Summary and Visualization Functions in multivar

Zachary F. Fisher

2026-01-25

Contents

1 Introduction **3**

2 Getting Started **3**
2.1 Loading the Package . . . 3
2.2 Simulating and Fitting a Model . . . 3

3 Model Summary: `summary_multivar()` **5**
3.1 Basic Usage . . . 5
3.2 Information Displayed . . . 5
3.2.1 Dataset Information . . . 5
3.2.2 Model Specification . . . 6
3.2.3 Adaptive Weights . . . 6
3.2.4 Overview of Dynamics . . . 6

4 Visualizing Dynamics: `print_dynamics()` **7**
4.1 Direct Use with Fit Objects . . . 7
4.2 Key Features . . . 7
4.3 Customization Options . . . 8

5 Time-Varying Parameter (TVP) Models **9**
5.1 Fitting a TVP Model . . . 9
5.2 TVP Model Summary . . . 10
5.3 TVP Dynamics with Custom Labels . . . 11

6 Edge Prevalence Across Subjects **13**
6.1 Proportion Display . . . 13
6.2 Count Display . . . 13
6.3 Interpreting the Output . . . 14

7 Comparison with $K = 3$ Model **15**

8 Complete Workflow Example **16**

9 Tips and Best Practices **16**
9.1 When to Use Each Function . . . 16
9.2 Customization Examples . . . 16

9.3 Working with Different Model Types 17
9.3.1 $K = 1$ Models (Single Subject) 17
9.3.2 $K > 1$ Models (Multiple Subjects) 17
9.3.3 TVP Models (Time-Varying) 17

10 Summary **18**

11 References **18**

1 Introduction

The `multivar` package provides three main functions for summarizing and visualizing fitted models:

1. **`summary_multivar()`** – Comprehensive model summary with dataset information, model specifications, and dynamics overview
2. **`print_dynamics()`** – Visualize dynamics matrices with clear formatting and time subscripts
3. **`print_edge_prevalence()`** – Display edge prevalence across subjects for multi-subject models ($K > 1$)

This vignette demonstrates how to use these functions to explore and interpret multivar model results effectively.

Key Features:

- Automatic detection of model types (TVP vs. non-TVP, $K = 1$ vs. $K > 1$)
- Clean, publication-ready formatting
- Flexible customization options
- Seamless integration with fitted model objects

---

2 Getting Started

First, load the package and fit a simple model to demonstrate the summary functions.

2.1 Loading the Package

```
library(multivar)
```

2.2 Simulating and Fitting a Model

We'll simulate data with $K = 2$ subjects, $d = 4$ variables, and $n = 100$ timepoints:

```
# Simulate data with K=2 subjects
set.seed(123)
sim <- multivar_sim(
  k = 2,
  d = 4,
  n = 100,
  prop_fill_com = 0.3,
  prop_fill_ind = 0.15,
  sigma = diag(4),
  lb = 0.1,
  ub = 0.9
)

# Fit the model
object <- constructModel(sim$data, intercept = FALSE)
#> Warning in constructModel(sim$data, intercept = FALSE): multivar WARNING:
```

```
#> common_tvp_effects = TRUE has no effect when tvp = FALSE.
fit <- cv.multivar(object)
```

3 Model Summary: summary_multivar()

The summary_multivar() function provides a comprehensive overview of your fitted model, including dataset characteristics, model specifications, and an overview of the estimated dynamics.

3.1 Basic Usage

```
summary_multivar(fit)
#>
#> ======================================================================
#> MULTIVAR Model Summary
#> ======================================================================
#>
#> Dataset Information:
#> ----------------------------------------------------------------------
#>   Number of datasets (subjects): 2
#>   Number of timepoints:          99 (all subjects)
#>   Number of variables:           4 (all subjects)
#>
#> Model Specification:
#> ----------------------------------------------------------------------
#>   Time-varying parameters:       NO (stationary)
#>   Intercepts:                    NO
#>
#> Adaptive Weights:
#> ----------------------------------------------------------------------
#>   Method:                        LASSO
#>   Type:                          Adaptive
#>   Lambda selection:              1 SE rule (lambda.1se)
#>   Cross-validation:              Blocked (10-fold)
#>
#> Overview of Dynamics:
#> ----------------------------------------------------------------------
#>   Common effects:  7 edges, 56.2% sparse, range [0.071, 0.638]
#>
#>   Total effects by subject:
#>     Subject 1:   9 edges, 43.8% sparse, range [-0.008, 0.638]
#>     Subject 2:   8 edges, 50.0% sparse, range [0.044, 0.696]
#>
#> ======================================================================
```

3.2 Information Displayed

The summary includes four main sections:

3.2.1 Dataset Information

- **Number of datasets (subjects)**: The K value

- **Number of timepoints**: Handles both equal and varying lengths across subjects
- **Number of variables**: The d value (dimension of the VAR system)

3.2.2 Model Specification

- **Time-varying parameters (TVP)**: Whether the model includes time-varying dynamics
- **Period ranges**: For TVP models, shows time ranges for each period
- **Intercepts**: Whether intercepts are included and how they are penalized

3.2.3 Adaptive Weights

- **Method**: LASSO, RIDGE, or other penalization methods
- **Type**: Adaptive or Standard weighting
- **Lambda selection**: Minimum CV error (`lambda.min`) or 1 SE rule (`lambda.1se`)
- **Cross-validation**: Type (blocked or rolling) and number of folds

3.2.4 Overview of Dynamics

- **Edge counts**: Number of non-zero entries in transition matrices
- **Sparsity**: Percentage of zero entries
- **Value ranges**: Range of non-zero coefficient values
- **Per-subject breakdown**: For $K > 1$ models, shows both common and subject-specific statistics

4 Visualizing Dynamics: print_dynamics()

The print_dynamics() function displays dynamics matrices with clear formatting, time subscripts, and summary statistics. A key feature is that you can pass fit objects directly.

4.1 Direct Use with Fit Objects

```
print_dynamics(fit, digits = 3)
#>
#> ======================================================================
#> FIT OBJECT: Total Effects
#> ======================================================================
#>
#> ======================================================================
#> Subject 1 Dynamics
#> ======================================================================
#>
#>    V1     V2     V3     V4
#> V1  0.473  0.071  0.242  0.294
#> V2  0.638 .      .      .
#> V3 -0.006 .      .       0.438
#> V4 .       0.278 .      -0.008
#>
#> Non-zero entries: 9 / 16 (43.8% sparse)
#> Range of non-zero values: [-0.008, 0.638]
#>
#>
#> ======================================================================
#> Subject 2 Dynamics
#> ======================================================================
#>
#>    V1     V2     V3     V4
#> V1  0.044  0.071  0.242  0.496
#> V2  0.638 .      .      .
#> V3  0.236 .      .      .
#> V4 .       0.278 .       0.696
#>
#> Non-zero entries: 8 / 16 (50.0% sparse)
#> Range of non-zero values: [0.044, 0.696]
```

4.2 Key Features

- **Time subscripts**: Rows labeled as $V_i(t)$, columns as $V_j(t-1)$
- **Dots for zeros**: Uses . to display zero entries for better readability
- **Alignment**: Positive values include leading space to align with negative values
- **Summary statistics**: Shows sparsity and value ranges
- **Automatic routing**: Detects TVP models and formats accordingly

4.3 Customization Options

```r
# More decimal places
print_dynamics(fit, digits = 4)

# Show zeros as "0" instead of "."
print_dynamics(fit, zero_char = "0")

# Without time subscripts
print_dynamics(fit, time_labels = FALSE)
```

5 Time-Varying Parameter (TVP) Models

The summary functions automatically detect and handle time-varying parameter models. Let's demonstrate with a TVP example.

5.1 Fitting a TVP Model

```r
# Create time-varying dynamics
phi_period1 <- matrix(c(
  0.3,  0.4,  0.0,  0.0,
  0.0,  0.5,  0.3,  0.0,
  0.4,  0.0,  0.2,  0.0,
  0.0,  0.3,  0.0,  0.4
), nrow = 4, byrow = TRUE)

phi_period2 <- matrix(c(
  0.4,  0.3,  0.0,  0.2,
  0.0,  0.6,  0.4,  0.0,
  0.3,  0.0,  0.3,  0.0,
  0.0,  0.4,  0.0,  0.5
), nrow = 4, byrow = TRUE)

phi_period3 <- matrix(c(
  0.5,  0.2,  0.0,  0.0,
  0.0,  0.4,  0.5,  0.0,
  0.5,  0.0,  0.4,  0.2,
  0.0,  0.5,  0.0,  0.3
), nrow = 4, byrow = TRUE)

# Generate data
phi_list <- c(
  rep(list(phi_period1), 80),
  rep(list(phi_period2), 80),
  rep(list(phi_period3), 80)
)

data1 <- multivar:::var_sim_growth(
  n = 240,
  phi = phi_list,
  sigma = diag(4),
  intercept = rep(0, 4)
)

# Fit TVP model
object_tvp <- constructModel(
  list(data1),
  intercept = FALSE,
  tvp = TRUE,
```

```
  breaks = list(c(80, 160)),
  cv = "blocked",
  nfolds = 5
)

fit_tvp <- cv.multivar(object_tvp)
```

5.2 TVP Model Summary

```
summary_multivar(fit_tvp)
#>
#> ======================================================================
#> MULTIVAR Model Summary
#> ======================================================================
#>
#> Dataset Information:
#> ----------------------------------------------------------------------
#>   Number of datasets (subjects): 1
#>   Number of timepoints:          239
#>   Number of variables:           4
#>
#> Model Specification:
#> ----------------------------------------------------------------------
#>   Time-varying parameters:       YES
#>   Number of periods:             3
#>   Period ranges:                 t=1-79, t=80-159, t=160-239
#>   Intercepts:                    NO
#>
#> Adaptive Weights:
#> ----------------------------------------------------------------------
#>   Method:                        LASSO
#>   Type:                          Adaptive
#>   Lambda selection:              1 SE rule (lambda.1se)
#>   Cross-validation:              Blocked (5-fold)
#>
#> Overview of Dynamics:
#> ----------------------------------------------------------------------
#>   Total effects (time-varying, 3 periods):
#>     Period 1:   9 edges, 43.8% sparse, range [0.022, 0.517]
#>     Period 2:   9 edges, 43.8% sparse, range [0.243, 0.583]
#>     Period 3:   8 edges, 50.0% sparse, range [0.291, 0.584]
#>
#> ======================================================================
```

Notice how the summary automatically detects the TVP structure and displays:

- Number of periods (3 in this case)
- Period ranges: $t = 1$–79, $t = 80$–159, $t = 160$–239

- Per-period edge statistics showing how network structure evolves over time

5.3 TVP Dynamics with Custom Labels

```
print_dynamics(
  fit_tvp,
  period_labels = c("Early", "Middle", "Late"),
  digits = 3
)
#>
#> ======================================================================
#> FIT OBJECT: Time-Varying Parameter Model
#> ======================================================================
#>
#> ======================================================================
#> TIME-VARYING DYNAMICS (3 periods)
#> ======================================================================
#>
#> ======================================================================
#> Period 1: Early
#> ======================================================================
#>
#>    V1     V2     V3     V4
#> V1  0.259  0.517 .      .
#> V2 .       0.494  0.258 .
#> V3  0.352 .       0.243 .
#> V4 .       0.389  0.022  0.291
#>
#> Non-zero entries: 9 / 16 (43.8% sparse)
#> Range of non-zero values: [0.022, 0.517]
#>
#>
#> ======================================================================
#> Period 2: Middle
#> ======================================================================
#>
#>    V1     V2     V3     V4
#> V1  0.259  0.417 .       0.257
#> V2 .       0.583  0.258 .
#> V3  0.352 .       0.243 .
#> V4 .       0.401 .       0.500
#>
#> Non-zero entries: 9 / 16 (43.8% sparse)
#> Range of non-zero values: [0.243, 0.583]
#>
#>
#> ======================================================================
```

```
#> Period 3: Late
#> ======================================================================
#>
#>    V1     V2     V3     V4
#> V1  0.434  0.352 .      .
#> V2 .       0.397  0.492 .
#> V3  0.584 .       0.317 .
#> V4 .       0.428 .       0.291
#>
#> Non-zero entries: 8 / 16 (50.0% sparse)
#> Range of non-zero values: [0.291, 0.584]
```

The period labels provide interpretable names for each time window, making it easier to communicate results.

6 Edge Prevalence Across Subjects

For models with multiple subjects ($K > 1$), `print_edge_prevalence()` shows which edges are present across subjects. This helps identify universal dynamics versus subject-specific patterns.

6.1 Proportion Display

Shows what proportion of subjects have each edge:

```
print_edge_prevalence(fit, type = "proportion", digits = 2)
#>
#> ======================================================================
#> Edge Prevalence (Proportion of Subjects)
#> ======================================================================
#>
#>    V1   V2   V3   V4
#> V1 1.00 1.00 1.00 1.00
#> V2 1.00 .    .    .
#> V3 1.00 .    .    0.50
#> V4 .    1.00 .    1.00
#>
#>   Universal edges (100%):        8
#>   Common edges (>50%):           8
#>   Rare edges (<50%):             0
#>   Absent edges (0%):             7
#> ======================================================================
```

6.2 Count Display

Shows the actual count of subjects with each edge:

```
print_edge_prevalence(fit, type = "count")
#>
#> ======================================================================
#> Edge Prevalence (Count of Subjects)
#> ======================================================================
#>
#>    V1  V2  V3  V4
#> V1 2/2 2/2 2/2 2/2
#> V2 2/2 .   .   .
#> V3 2/2 .   .   1/2
#> V4 .   2/2 .   2/2
#>
#>   Universal edges (all 2):       8
#>   Variable edges (between 1-1):  1
#>   Absent edges (0):               7
#> ======================================================================
```

6.3 Interpreting the Output

The summary statistics identify:

- **Universal edges (100%)**: Present in all subjects – these represent shared dynamics
- **Common edges ($> 50\%$)**: Present in more than half of subjects
- **Rare edges ($< 50\%$)**: Present in less than half of subjects – potentially subject-specific
- **Absent edges (0%)**: Not present in any subject

This breakdown helps researchers understand:

1. **Shared network structure**: Universal and common edges
2. **Individual variation**: Rare edges showing subject-specific patterns
3. **Network sparsity**: Absent edges indicate where no relationships exist

7 Comparison with $K = 3$ Model

Edge prevalence patterns become more nuanced with additional subjects:

```
# Simulate K=3 model
sim3 <- multivar_sim(
  k = 3,
  d = 4,
  n = 100,
  prop_fill_com = 0.3,
  prop_fill_ind = 0.15,
  sigma = diag(4),
  lb = 0.1,
  ub = 0.9
)

object3 <- constructModel(sim3$data, intercept = FALSE)
#> Warning in constructModel(sim3$data, intercept = FALSE): multivar WARNING:
#> common_tvp_effects = TRUE has no effect when tvp = FALSE.
fit3 <- cv.multivar(object3)
```

```
print_edge_prevalence(fit3, type = "proportion", digits = 2)
#>
#> ======================================================================
#> Edge Prevalence (Proportion of Subjects)
#> ======================================================================
#>
#>    V1 V2   V3   V4
#> V1 .  0.33 0.33 1.00
#> V2 .  1.00 0.33 1.00
#> V3 .  .    .    0.33
#> V4 .  1.00 1.00 0.33
#>
#>   Universal edges (100%):        5
#>   Common edges (>50%):           5
#>   Rare edges (<50%):             5
#>   Absent edges (0%):             6
#> ======================================================================
```

With more subjects, you can observe more granular patterns of edge prevalence, helping to distinguish between highly consistent edges, moderately consistent edges, and idiosyncratic edges.

8 Complete Workflow Example

A typical workflow combining all three functions:

```r
# 1. Fit your model
fit <- cv.multivar(object)

# 2. Get comprehensive summary
summary_multivar(fit)

# 3. Visualize dynamics
print_dynamics(fit)

# 4. For K>1: Check edge prevalence
print_edge_prevalence(fit, type = "proportion")
```

---

9 Tips and Best Practices

9.1 When to Use Each Function

- **`summary_multivar()`**: Always use this first to understand model structure and get an overview
- **`print_dynamics()`**: Use to examine specific dynamics matrices with publication-ready formatting
- **`print_edge_prevalence()`**: Use for $K > 1$ models to understand shared vs. unique network structure

9.2 Customization Examples

All functions support flexible customization:

```r
# Summary (automatic - no customization needed)
summary_multivar(fit)

# Dynamics with custom formatting
print_dynamics(
  fit,
  digits = 4,              # More precision
  zero_char = "-",         # Custom zero character
  period_labels = c(...)   # For TVP models
)

# Edge prevalence with custom display
print_edge_prevalence(
  fit,
  type = "count",          # Show counts instead of proportions
  digits = 3,              # Decimal places for proportions
  zero_char = "-"          # Custom zero character
)
```

9.3 Working with Different Model Types

9.3.1 $K = 1$ Models (Single Subject)

- `summary_multivar()` shows single-subject statistics
- `print_dynamics()` displays one matrix
- `print_edge_prevalence()` returns a message (not applicable for single subject)

9.3.2 $K > 1$ Models (Multiple Subjects)

- `summary_multivar()` shows common + subject-specific statistics
- `print_dynamics()` displays matrices for all subjects
- `print_edge_prevalence()` shows edge overlap across subjects

9.3.3 TVP Models (Time-Varying)

- `summary_multivar()` shows per-period statistics
- `print_dynamics()` displays time-varying matrices with period labels
- `print_edge_prevalence()` shows prevalence separately for each period (if $K > 1$)

10 Summary

The multivar summary functions provide:

- **Comprehensive information** about model structure and results
- **Clear visualization** of dynamics with readable, publication-ready formatting
- **Cross-subject comparisons** via edge prevalence analysis
- **Automatic detection** of model types (TVP, $K = 1$ vs. $K > 1$)
- **Flexible customization** for different display and reporting needs

These tools make it easy to explore, interpret, and communicate multivar model results in both exploratory data analysis and formal research reporting.

---

11 References

Fisher, Z. F., Kim, Y., Fredrickson, B. L., & Pipiras, V. (2022). Penalized estimation of multiple-subject Vector Autoregressive models. *Psychometrika*, *87*(1), 1–28.